

Computer Science

Computational Thinking Exercise

Karmela Liane A. Oblina

9 - Beryllium

Annex A

Computational Thinking Exercise: "Smart School Canteen Queue"

Section: 9 - Beryllium

Score:

C# / Name: Karmela Liane A. Oblina

Date: August 11, 2026

Scenario

The PSHS school canteen is small and often gets crowded during lunch break. Students line up to buy food, but the process is slow because:

- *Some students take too long to decide what to order.*
- *The cashier has to manually calculate totals and give change.*
- *There is no system to track which food items are running out.*

Your group's task is to decompose this problem into smaller, manageable parts that could be solved with computational thinking (CT) Skills.

Step 1: Identify the Big Problem

Main Problem:

The PSHS school canteen suffers a big problem from overcrowding, especially during lunch break, and slow service due to indecisive students, inefficient ordering, manual payment processing, and lack of real-time tracking of items.

Step 2: Identify three to four Sub-Problems

Please list possible sub-problems:

1. Delayed Decision-Making at the Counter

2. Slow Manual Computation of Payment During Checkout

3. Lack of Real-Time Tracking of Items

4. Overcrowding and Poor Queue Management

Step 3: Define Computational Thinking Approaches

For each sub-problem, apply CT skills:

Sub-Problem	CT Skill	Example Solution
Delayed Decision-Making at the Counter	Abstraction - The abstraction skill helps prevent delayed decision-making at the counter by focusing only on the most important information needed to make a decision. It reduces confusion and mental overload, allowing staff to respond to customers quickly and confidently	Create a physical or digital menu near the counter and along the outer part of where the queue will be. The menu should be composed of the items correctly categorized, its prices, and visual icons. Irrelevant details, such as ingredients, should be avoided as it would lead to longer decision-making. Therefore, students can make rapid choices before reaching the counter.
Slow Manual Computation of Payment During Checkout	Algorithm Design - Algorithm Design helps prevent slow manual computation of payment during checkout by providing a clear, step-by-step process for calculating the total, discounts, and change. This makes computations faster and more accurate, reducing errors and shortening customer waiting time.	Build a sequential algorithm for a simple checkout interface for the cashiers. The algorithm is simply: 1. Choose selected items 2. Calculate sum of items 3. Input cash received 4. Calculate change (Cash - Sum) 5. Display output 6. Print Receipt

Lack of Real-Time Tracking of Items	Pattern Recognition - Pattern Recognition helps address the lack of real-time tracking of items by identifying patterns in inventory movement, such as frequently sold or low-stock items. This allows staff to monitor item availability more efficiently and quickly recognize when stock needs to be updated or replenished	Track historical data to recognize patterns of which items sell out the fastest during the first half hour of the lunch period. Using these patterns, establish a digital notification to alarm staff to restock or take off a specific item in the menu. Additionally, establish an automated digital rule that if the stock level is less than 5 units, send an alert to kitchen staff to prepare more.
Overcrowding and Poor Queue Management	Decomposition - Decomposition helps address overcrowding and poor queue management by breaking the problem into smaller parts, such as identifying customer flow, waiting areas, and service times. This makes it easier to find the cause of overcrowding and organize the queue more efficiently.	Break the large queues and overcrowding problem by: 1. Making sure that there are at least three cashiers in every counter, ensuring that the queues will be broken down into shorter ones. 2. Another three lanes for picking up the food and at least 2 staff are assigned to each lane. 3. The menus are placed along the queues so students can pick while they are in line. The same menus can also be placed in another corner, big enough so that overcrowding won't occur.

Step 4:

Selected Sub-Problem: Lack of Real-Time Tracking of Items

PSEUDOCODE

START

Set total_sum = 0

Set items_list = empty list

LOOP

 Ask user for item_name

 IF item_name is empty THEN

 BREAK LOOP

 ENDIF

 Ask user for item_price

 Add item_price to total_sum

 Add (item_name, item_price) to items_list

ENDLOOP

Display total_sum

LOOP

 Ask user for cash_received

 IF cash_received >= total_sum THEN

 BREAK LOOP

 ELSE

 Display "Insufficient payment error"

 ENDIF

ENDLOOP

Compute change = cash_received - total_sum

Display "Payment Accepted"

Display change

Display receipt header

FOR EACH item IN items_list

 Display item.name and item.price

ENDFOR

Display total_sum

Display cash_received

Display change

Display receipt footer

END